DATA ANALYSIS PROJECT 01

Insurance Premiums and GDP per capita

Heejung Heo

November 2026

Contents

1. Objectives & Approach
2. Data Missingness Check
3. Data Imputation
4. Data Analysis
5. Limitations & Suggestions

Objectives & Approach

- Examine trends in insurance premiums and GDP across countries
- Explore the relationship between insurance premiums and GDP per capita

(based on OECD Insurance Statistics 2022)

01. Dataset

Collect Dataset

[OECD]
Premiums, claims and expenses of insurance companies

[World Bank]
1. GDP per capita, PPP (constant 2021 International $)
2. Population, total

02. Data Preparation

Missingness Check

1. Check missing values by country and year
2. Calculate missing rates for 1995 – 2024
3. Select 30 countries with combined – premium missing rates below 30%

Imputation

[Internal gaps]
- Linear interpolation
- Kalman smoothing

[Leading / trailing gaps]
- Naive
- ETS
- ARIMA

▶ Select methods using validation RMSE

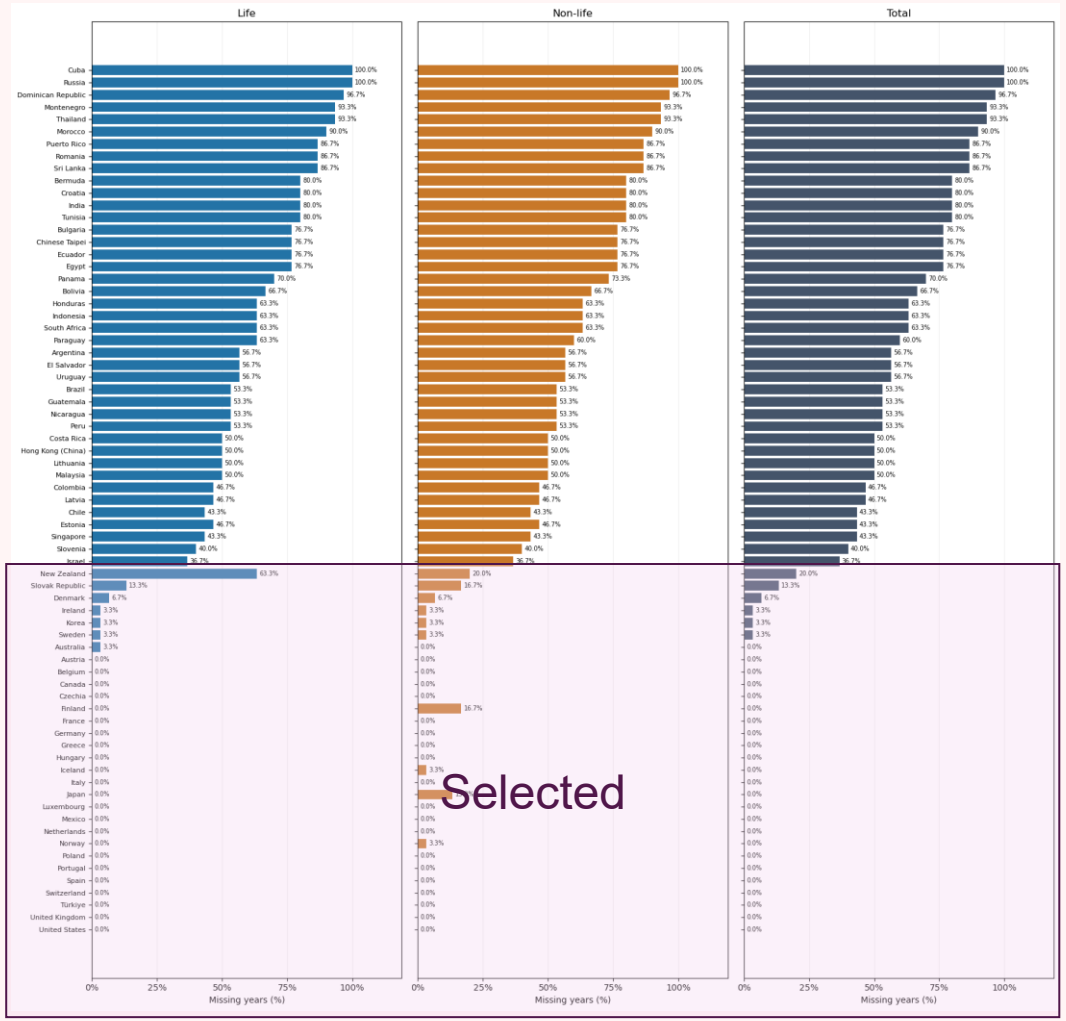
03. Analysis

Statistical Analysis

1. Compare long-term trends (1995-2024)
2. Assess correlations
   - Pearson
   - Spearman
3. Compare regression results for 1995 and 2024

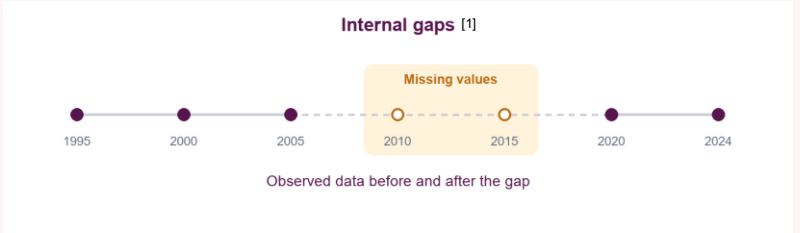
Data Missingness Check

- Check data availability by country and year
- Calculate the proportion of years for which neither life nor non-life premium data are available
- Apply a missing-rate threshold of <30% to select countries
- Related code:
  02_missingness_and_country_selection.ipynb



Data Imputation 1 – Internal gaps

- Internal gaps are missing values with observed data both before and after them. Earlier data from before 1995 are also used where available

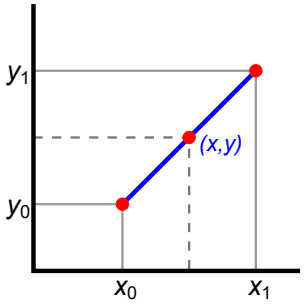


Linear Interpolation

[Assumption]
Values increase or decrease at a constant rate between two observed endpoints

[Method]
Connect the two endpoints with a straight line to estimate the missing values between them

[2]



[Code Snippet]

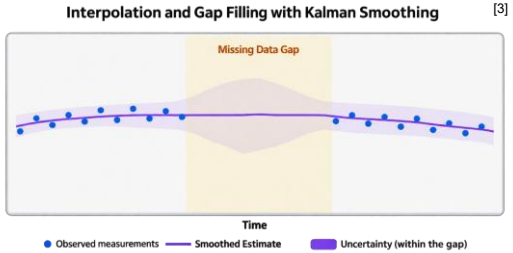
```
def predict_linear(values, gap_positions):
    observed_positions = np.flatnonzero(np.isfinite(values))
    return np.interp(gap_positions, observed_positions, values[observed_positions])
```

Kalman smoothing

[Assumption]
Values follow an underlying trend that can change over time, with random fluctuations

[Method]
Use observations before and after the gap to estimate missing values with a local linear trend model

[3]

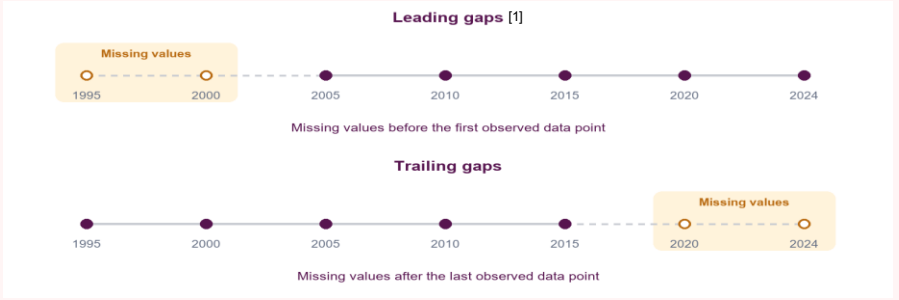


[Code Snippet]

```
def predict_kalman(values, gap_positions):
    center = np.nanmean(values)
    scale = np.nanstd(values)
    if scale < 1e-12:
        return np.repeat(center, len(gap_positions))
    scaled_values = (values - center) / scale
    model = UnobservedComponents(scaled_values, level="local linear trend")
    fitted_model = model.fit(disp=False, maxiter=200)
    if not fitted_model.mle_retvals.get("converged", False):
        return None
    return center + scale * fitted_model.smoothed_state[0, gap_positions]
```

[1] AI-generated image, [2] Wikipedia “Linear interpolation”, [3] Kalman Filters and Smoothing for Time-Series Data, Aayush Garg

Data Imputation 2 – Leading / trailing gaps



- Leading gaps : Missing values before the first observed data point
- Trailing gaps : Missing values after the last observed data point

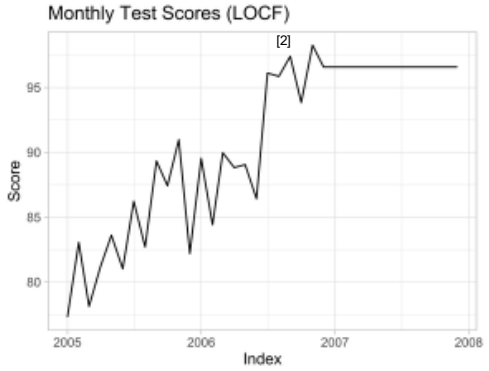
Naive

[Assumption]

Values remain constant beyond the observed period

[Method]

- Fill missing values by repeating the nearest observed value
- Forward fill for trailing gaps (LOCF) + Backward fil for leading gaps (BOCF)



[Code Snippet]

- See Appendix A.

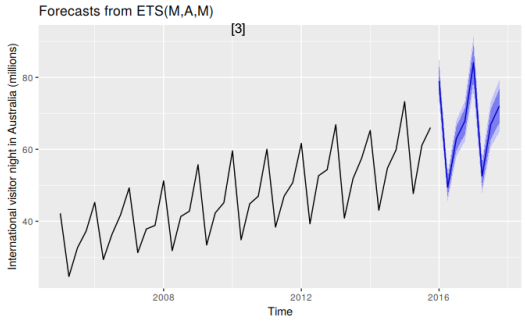
ETS

[Assumption]

- Recent patterns in the data continue beyond the observed period
- Error + Trend + Seasonal / Exponential smoothing

[Method]

Give more weight to recent observations and use the estimated level and trend to predict missing values



[Code Snippet]

- See Appendix A.

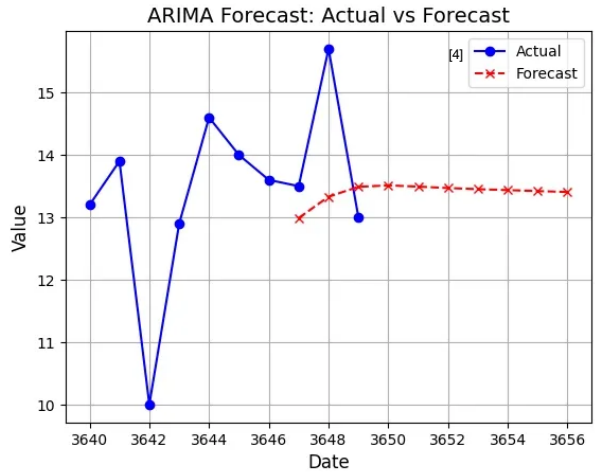
ARIMA

[Assumption]

Past relationships remain stable after differencing, if needed

[Method]

Use past values and forecast errors to predict missing values



[Code Snippet]

- See Appendix A.

[1] AI-generated image, [2] Datacamp "Imputing missing values.mp4", [3] Forecasting: Principles and Practice, Rob J Hyndman, [4] geeksforgeeks, "ARIMA"

Data Imputation 3 – Select methods using RMSE (1/3)

- Root Mean Squared Error measures the difference between predicted and observed values.
- Lower RMSE indicates smaller prediction errors

$$RMSE = \sqrt{\sum_{i=1}^{n} \frac{(\hat{y}_i - y_i)^2}{n}}$$

[Processes (1)]

1. Temporarily hide consecutive known values matching the actual gap's length and type

```
def find_validation_blocks(values, gap_length, gap_type, reported_breaks):
    blocks = []
    for start in range(len(values) - gap_length + 1):
        positions = np.arange(start, start + gap_length)
        end = positions[-1]
        if np.isnan(values[positions]).any() or reported_breaks[positions].any():
            continue
        if np.isfinite(np.delete(values, positions)).sum() < minimum_observations:
            continue
        if gap_type == "internal":
            if start == 0 or end == len(values) - 1:
                continue
            if np.isnan(values[start - 1]) or np.isnan(values[end + 1]) or reported_breaks[end + 1]:
                continue
        else:
            if gap_type == "leading":
                if end == len(values) - 1 or np.isnan(values[end + 1]) or reported_breaks[end + 1]:
                    continue
            elif start == 0 or np.isnan(values[start - 1]):
                continue
            if get_adjacent_history(values, positions, gap_type) is None:
                continue

        blocks.append(positions)
    return blocks
```

Data Imputation 3 – Select methods using RMSE (2/3)

[Processes (2 – 5)]

2. Predict the hidden values using each candidate method
   - Internal gaps: Linear Interpolation vs Kalman Smoothing
   - Leading/Trailing gaps: Naïve vs ETS vs ARIMA

3. Repeat across 3 – 6 validation blocks and collect the observed and predicted values

4. Calculate the overall RMSE across all validation blocks

5. Predict the actual gap, check validity and return the candidate's predictions and validation scores

```python
def evaluate_method(values, gap_positions, validation_blocks, method, gap_type):
    actual_values = []
    predicted_values = []
    messages = []
    for positions in validation_blocks:                                   # Process 3
        hidden_values = values.copy()
        hidden_values[positions] = np.nan
        if gap_type == "leading":
            hidden_values[:positions[0]] = np.nan
        if gap_type == "trailing":
            hidden_values[positions[-1] + 1:] = np.nan
        prediction, message = predict_gap(hidden_values, positions, method, gap_type)   # Process 2
        if prediction is None:
            return None, np.nan, np.nan, message
        actual_values.extend(values[positions])
        predicted_values.extend(prediction)
        if message:
            messages.append(message)
    errors = np.array(predicted_values) - np.array(actual_values)             # Process 4
    rmse = np.sqrt(np.mean(errors ** 2))
    mae = np.mean(np.abs(errors))
    prediction, message = predict_gap(values, gap_positions, method, gap_type)   # Process 5
    if prediction is None:
        return None, np.nan, np.nan, message
    if message:
        messages.append(message)
    return prediction, rmse, mae, "; ".join(dict.fromkeys(messages))
```

Data Imputation 3 – Select methods using RMSE (3/3)

[Processes (6 - 7)]

6. Choose the method and available years of data that give the lowest validation RMSE

```
valid = [number for number, row in enumerate(candidate_rows) if np.isfinite(row["RMSE"])]
if not valid:
    return None, candidate_rows
lowest_rmse = min(candidate_rows[number]["RMSE"] for number in valid)
tolerance = max(1e-6, lowest_rmse * 1e-8)
tied = [number for number in valid if candidate_rows[number]["RMSE"] <= lowest_rmse + tolerance]
```

```
def tie_priority(number):
    row = candidate_rows[number]
    simple = row["Method"].startswith(("linear", "naive"))
    return 10 * int(not simple) + int(row["History"] == "Full history")
best_number = min(tied, key=tie_priority)
candidate_rows[best_number]["Selected"] = True
best_result = candidate_rows[best_number].copy()
best_result["values"] = predictions[best_number]
return best_result, candidate_rows
```

7. Fill the actual gap using the selected method or leave it unfilled if no candidate meets the requirements

```
if np.isfinite(values).sum() == 0:
    reason = "Unresolved: no observed values; applicability unknown"
elif np.isfinite(values).sum() < minimum_observations:
    reason = "Unresolved: too few observations"
next_position = gap_positions[-1] + 1
if reported_breaks[gap_positions].any() or (
    next_position < len(values) and reported_breaks[next_position]
):
    reason = "Unresolved: reported break at gap"
best_result = None

if not reason:
    best_result, candidates = choose_gap_method(values, gap_positions, gap_type, reported_breaks)
    for row in candidates:
        row.update(Country=country_name, Type=insurance_type, Business=business_type,
                   Gap=gap_label, Gap_type=gap_type)
        validation_rows.append(row)
    if best_result is None:
        reason = "Unresolved: insufficient validation blocks or failed candidates"
```

```
if best_result is not None:
    result.loc[gap_positions, "Filled"] = best_result["values"]
    result.loc[gap_positions, "Is_imputed"] = True
    for column in ["Method", "History", "Blocks", "RMSE", "MAE"]:
        result.loc[gap_positions, column] = best_result[column]
    reason = "Imputed: matched-block comparison"
result.loc[gap_positions, "Status"] = reason
```

Data Imputation 3 – Results

- Unresolved missing values reduced the sample from 30 to 28 countries for the GWP analysis and to 27 countries for the direct-premium comparison

Country	Excluded from	Reason
Slovakia, 1995 - 1996	Both analyses	Imputation also used data from before 1995. However, a reported data break in 2009 restricted usable data to 2008 or earlier. Only 10 observations were available, which is below our minimum requirement of 12. Thus, the missing values remained unfilled.
New Zealand, 2022 - 2024	Both analyses	No eligible three-year trailing validation blocks are available for the main direct-business premium components or Non-life accepted reinsurance. Other components have too few observations. Thus, the missing values remained unfilled.
United Kingdom, 1995 (Life-direct)	Direct-premium analyses	The direct-Life gap covers 1983-1995. Its break-restricted 1983-2008 context has 13 observations but no eligible matched blocks for this 13-year gap. Accepted reinsurance can be estimated separately. Thus, GWP availability does not establish direct-premium availability.

Data Analysis – long-term trends (Insurance premium, GDP per capita) 1

[Indicators]

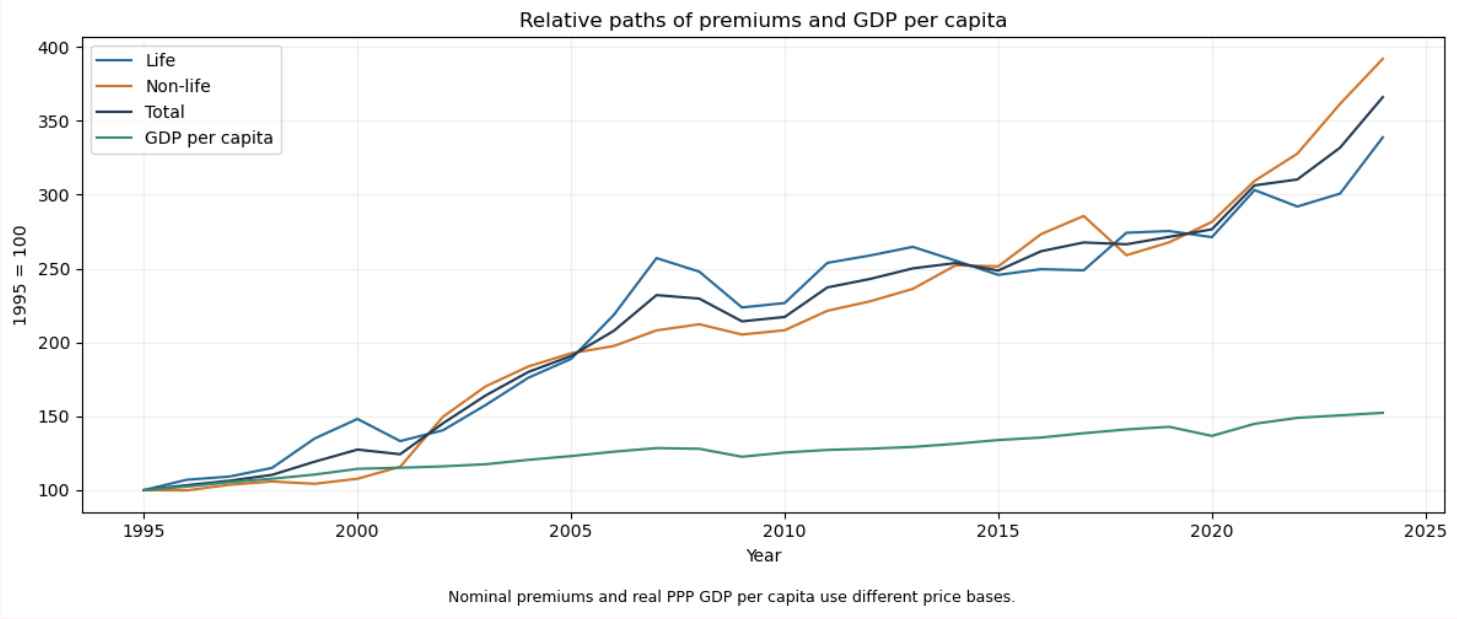
$$\overline{GDP_{pc_t}} = \frac{\sum_{i \in C} GDP_{pc_{it}} \times Population_{it}}{\sum_{i \in C} Population_{it}},$$

$$Index_t = \frac{Value_t}{Value_{1995}} \times 100$$

i : country
t : year
C : fixed set of 28 countries

[Summary]

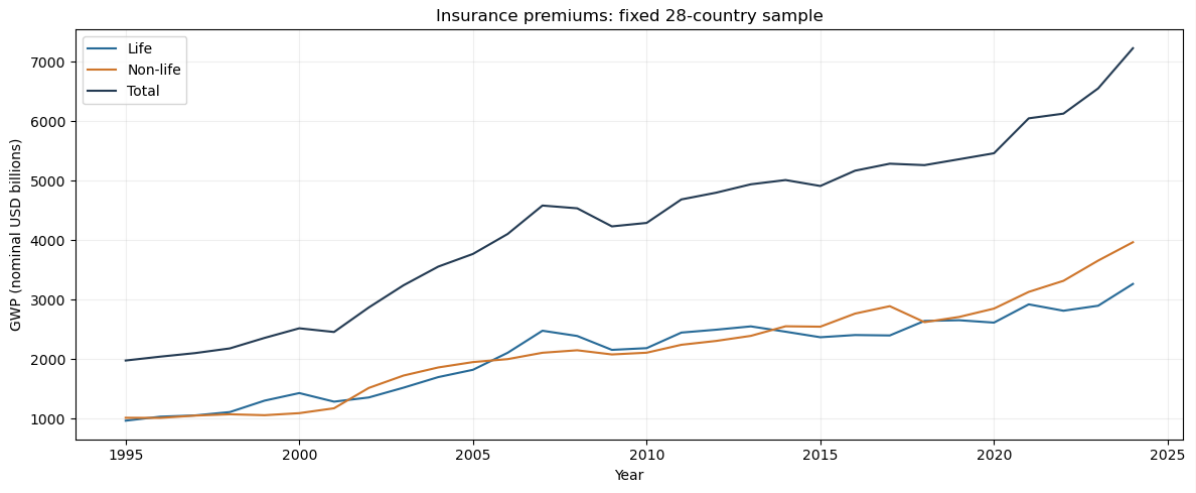
- Non-life premiums show the largest cumulative increase, while Life premiums show more pronounced fluctuations
- Premium indices rise more sharply after 2020. However, nominal aggregate premiums and real GDP per capita are not directly comparable measures of growth.

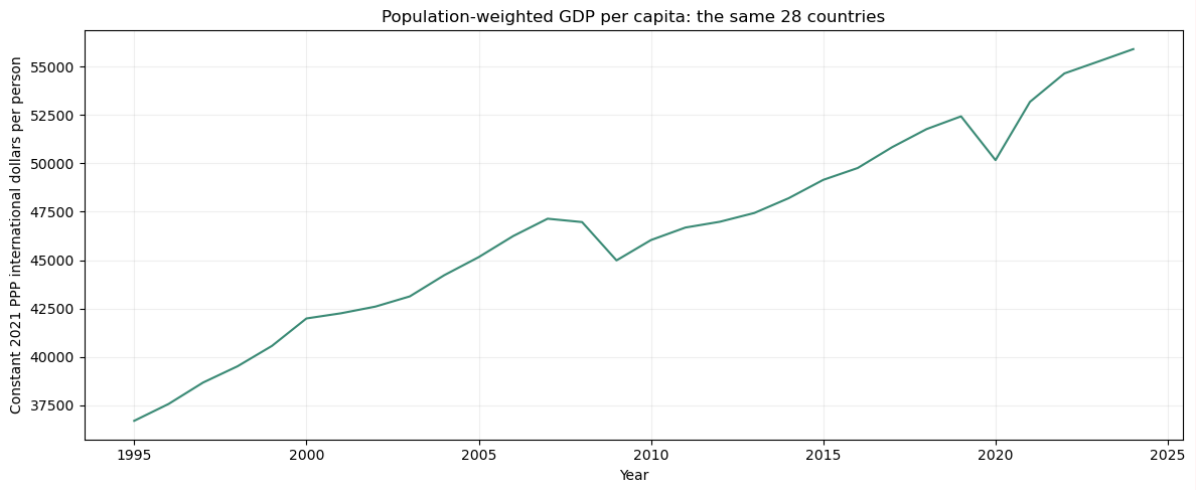


Data Analysis – long-term trends (Insurance premium, GDP per capita) 2

[Summary]

- Insurance premiums and GDP per person generally increased from 1995 to 2024.
- Life insurance premiums show larger ups and downs, while Non-life premiums rise more steadily.
- Non-life premiums exceed Life premiums by 2024.
- Both total premiums and GDP per person fall around 2009, then recovered.
- GDP per person fell in 2020 during the COVID-19 pandemic, then recovered





Premiums are total amounts before adjusting for inflation. GDP per person is adjusted for inflation. Their growth rates are therefore not directly comparable.

Data Analysis – Correlations (Pearson, Spearman) 1

[Question]

- Do countries with higher GDP per person also have higher insurance premiums, both in total and per person?

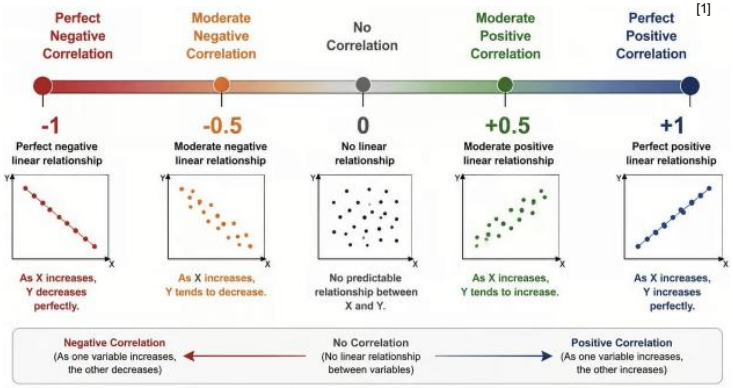
[Pearson & Spearman]

Pearson correlation coefficient

$$\rho_{X,Y} = \frac{E[(X - \mu_X)(Y - \mu_Y)]}{\sigma_X \sigma_Y}$$

[Interpretation]

- Measures linear correlation between two sets of data
- Values range from -1 to +1
- In this study, Pearson correlation is calculated using log-transformed GDP per capita and premium values



[1]

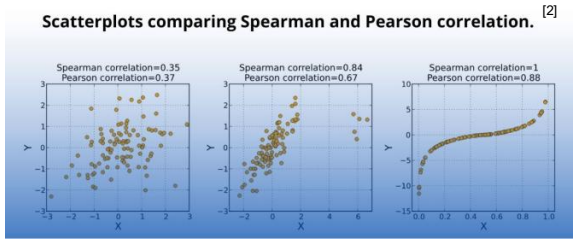
Spearman's rank correlation coefficient

$$r_s = \rho\big[R[X], R[Y]\big] = \frac{cov\,[R[X], R[Y]]}{\sigma_{R[X]}\sigma_{R[Y]}},$$

$$R[X], R[Y] : ranks\ of\ (X, Y)$$

[Interpretation]

- Measures the strength and direction of relationship between two variables based on their ranks
- Values range from -1 to +1
- In this study, spearman's rank correlation gives answer to "Do countries with higher GDP per person also rank higher in insurance premiums?"



[2]

[1] EARLY YEARS TV "What is a correlation coefficient? Pearson's r and Psychology statistics" [2] LEARN STATISTICS EASILY "Understanding Spearman Correlation in Data Analysis"

Data Analysis – Correlations (Pearson, Spearman) 2

[Indicators]

$$GWP_{pc_{it}} = GWP_{it}/Population_{it}\,,\ Direct_{pc_{it}} = Direct\ Premium_{it}/Population_{it}$$

i : country
t : year

* GWP : direct premiums + accepted reinsurance, pc : per person

[Summary]

* The same 27 countries in both years

		Pearson (log-scale)			Spearman		
Year	**Type**	**GWP**	**GWP$_{pc}$**	**Direct$_{pc}$**	**GWP**	**GWP$_{pc}$**	**Direct$_{pc}$**
1995	**Life**	0.382	0.819	0.812	0.477	0.688	0.685
	Non-life	0.292	0.890	0.896	0.400	0.891	0.889
	Total	0.365	0.888	0.885	0.456	0.795	0.784
2024	**Life**	0.210	0.765	0.762	0.284	0.708	0.708
	Non-life	0.129	0.889	0.879	0.175	0.896	0.910
	Total	0.197	0.891	0.883	0.266	0.866	0.868

- Countries with higher GDP per person do not necessarily have higher total insurance premiums
- The relationship is much stronger when premiums are measured per person
- Premiums per person show a strong positive relationship with GDP per person in both 1995 and 2024
- Pearson and Spearman show the same overall pattern

Data Analysis – Compare regression results (1995 vs 2024)

[Models]

$$\text{B1}: \log(GWP_i) = \alpha + \beta \log\left(GDP_{pc_i}\right) + \epsilon_i$$

$$\text{B2}: \log(GWP_i) = \alpha + \beta \log\left(GDP_{pc_i}\right) + \gamma \log(Population_i) + \epsilon_i$$

[Summary]

* Combined Life and Non-life GWP across 28 countries

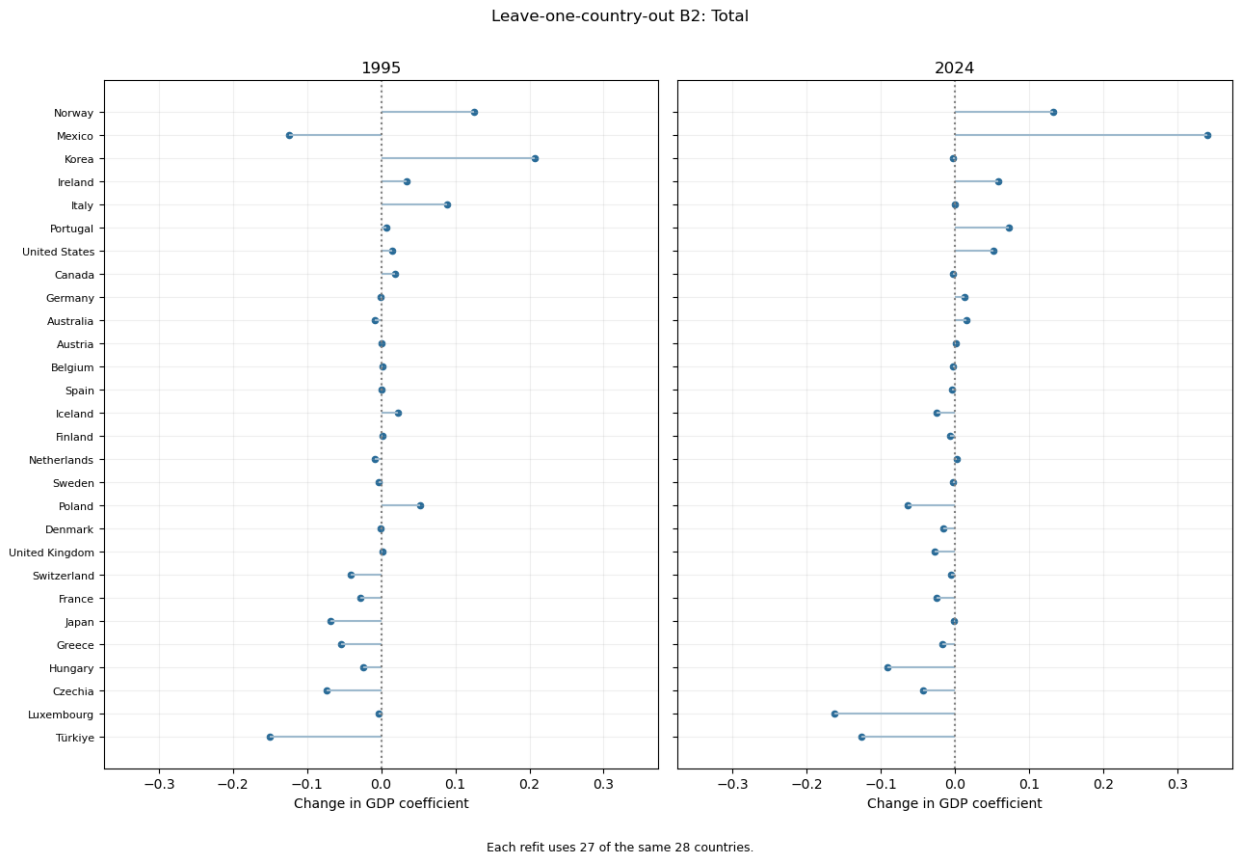
Year	Model	Log predictor	Estimate	95% CI
1995	B1	GDPpc	1.543	[-0.058, 3.145]
	B2	GDPpc	3.167	[2.436, 3.897]
		Population	1.156	[1.015, 1.297]
2024	B1	GDPpc	0.855	[-0.701, 2.412]
	B2	GDPpc	3.401	[2.470, 4.332]
		Population	1.215	[1.089, 1.342]

- Without accounting for population :
  The relationship between GDP per person and total premium is unclear in both 1995 and 2024.
- After accounting for population :
  Countries with higher GDP per person tend to have higher total premiums in both 1995 and 2024.
- A 1% higher GDP per person is associated with approximately 3.17% higher premiums in 1995 and 3.40% in 2024, holding population constant.

Data Analysis – Leave-one-country-out B2 sensitivity test

[Summary]

- Repeated the regression after excluding one country at a time.
- Removing individual countries makes the GDP coefficient higher or lower.
- However, the coefficient remains positive in every case, in both 1995 and 2024.
- No single country explains the positive relationship between GDP per person and total premiums after accounting for population.



Limitations & Suggestions

Limitations	Suggestions
Different price bases: Premiums are in nominal USD, while GDP per person is adjusted for inflation	Adjust premiums for inflation and exchange-rate effects to compare real changes over time
Differences across countries: The regressions compare countries within each year. They do not show the results as income changes within a country	Use a panel model to examine whether premiums rise as GDP per person increases within the same country
Overlapping premium coverage: GWP includes accepted reinsurance and premiums per person do not equal spending by an average resident	Focus on direct premiums and where available, Separate household and business insurance
Uncertainty from estimated values: Some missing values were filled, but this uncertainty is not included in the confidence intervals	Repeat the analysis using several plausible sets of imputed values and compare the results

Appendix A – Code Snippet

- Related code: 03_imputation.ipynb

Naive

```
if method_name == "naive" or scale < 1e-12:
    prediction = np.repeat(training[-1], gap_length)
else:
    scaled_training = (training - center) / scale
    if method_name == "ets":
        prediction = predict_ets(scaled_training, gap_length)
    elif method_name == "arima":
        prediction = predict_arima(scaled_training, gap_length)
    else:
        raise ValueError("Unknown method.")
    prediction = center + scale * prediction

if gap_type == "leading":
    prediction = prediction[::-1]
```

ETS

```
def predict_ets(training_values, horizon):
    best_aicc = np.inf
    best_prediction = None
    for trend, damped in [(None, False), ("add", False), ("add", True)]
    :
        try:
            model = ETSModel(training_values, error="add", trend=trend,
                             damped_trend=damped, seasonal=None)
            fitted_model = model.fit(disp=False, maxiter=300)
            if not fitted_model.mle_retvals.get("converged", False):
                continue
            if np.isfinite(fitted_model.aicc) and fitted_model.aicc <
            best_aicc:
                best_aicc = fitted_model.aicc
                best_prediction = np.asarray(fitted_model.forecast
                (horizon))
        except (ValueError, np.linalg.LinAlgError):
            continue

    if best_prediction is None:
        raise ValueError("No valid ETS model.")

    return best_prediction
```

- This study compared non-seasonal models with no trend, an additive trend or a damped trend.
- Selected the model using AICc.

ARIMA

```
def predict_arima(training_values, horizon):
    kpss_pvalue = kpss(training_values, regression="c", nlags="auto")
    [1]
    differences = int(kpss_pvalue < 0.05)
    trend_options = ["n", "c"] if differences == 0 else ["n", "t"]
    best_aicc = np.inf
    best_prediction = None

    for ar_order in [0, 1]:
        for ma_order in [0, 1]:
            for trend in trend_options:
                try:
                    model = ARIMA(training_values, order=(ar_order,
                    differences, ma_order), trend=trend)
                    fitted_model = model.fit(method_kwargs={"maxiter":
                    150})
                    if not fitted_model.mle_retvals.get("converged",
                    False):
                        continue
                    if np.isfinite(fitted_model.aicc) and fitted_model.
                    aicc < best_aicc:
                        best_aicc = fitted_model.aicc
                        best_prediction = np.asarray(fitted_model.
                        forecast(horizon))
                except (ValueError, np.linalg.LinAlgError):
                    continue

    if best_prediction is None:
        raise ValueError("No valid ARIMA model.")

    return best_prediction
```